

Table des matières

Arbre binaire ordonné	2
Définition	2
Propriétés	3
Recherche dans un arbre binaire de recherche	4
Insertion dans un arbre binaire ordonné	8
Problème	8
Fonction inserABR : TElement x Arbre → Arbre	8
Méthode 1 : insertion en feuille	8
Méthode 2 : version non récursive d'insertion en feuille	10
Méthode	10
Principe	10
Recherche de la Place d'insertion	12
Exemples	12
Schéma récursif de la Recherche de la Place d'insertion	13
Algorithme réalisant l'insertion	15
Méthode 2 : Insertion à la racine	16
Intérêt	16
Principe	16
Exemple d'insertion en racine de 14	18
Remarques	19
Application : tri d'un tableau	22
Principe	22
Algorithme de tri	24
Complexité	26
Algorithmes de Recherche du plus petit (resp. Plus Grand) élément dans un ABR	31
Principe	31
Rechercher l'élément le plus à gauche	32
Recherche l'adresse du plus petit (plus grand) élément dans un ABR	34
Algorithmes de suppressions	36
Suppression du nœud racine	36
Principe	36
Suppression de la racine on distingue les cas suivants	36
Exemples	37
Version2 suppression racine	39
Suppression d'un élément dans un arbre binaire ordonné	40
Problème	40
Exemples : voir CM	41
Principe	43
Changement de la suppression racine	45
Suppression de l'élément max dans un arbre binaire de recherche	46
Exercice	48
Intérêts des ABR	49

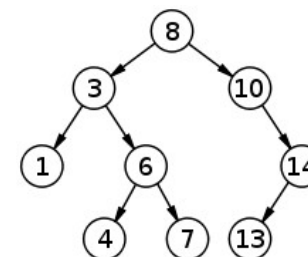
Arbre binaire ordonné

Arbre binaire ordonné = arbre binaire de recherche

Soit ($\leq$) une relation d'ordre sur l'ensemble des valeurs (données) attachées aux nœuds d'un arbre binaire.

Définition

Un arbre binaire est dit ordonné si la chaîne infixée des valeurs correspondant au parcours infixé est ordonnée.

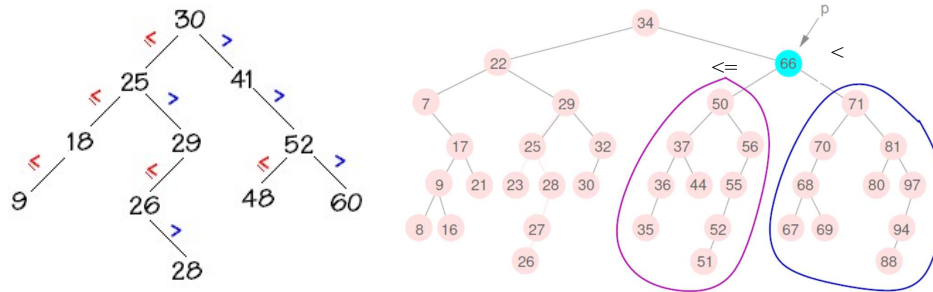
**Rappel du Parcours Infixe**

Ce parcours consiste à effectuer dans l'ordre :

- Parcours du fils gauche
- Traitement de la racine
- Parcours du fils droit

Le parcours infixé de l'arbre donne une liste ordonnée de valeurs :

[1, 3, 4, 6, 7, 8, 10, 13, 14]



Propriétés

Un arbre binaire de recherche A est un arbre qui possède les propriétés suivantes :

Si e est la donnée associée au nœud A, alors toutes les données associées au FilsGauche(A) sont inférieures ou égale ($\leq$) à e et tous les données associées au FilsDroit(A) sont supérieurs ($>$) à e.

Autrement dit : chaque nœud de l'arbre A vérifié les propriétés suivantes :

Si les deux fils du nœud A sont non vides :

$$\text{donnee}(\text{filsGauche}(A)) \leq \text{donnee}(A) < \text{donnee}(\text{filsDroit}(A))$$

Si le fils gauche est vide : $\text{donnee}(A) < \text{donnee}(\text{filsDroit}(A))$

Si le fils droit est vide : $\text{donnee}(\text{filsGauche}(A)) \leq \text{donnee}(A)$

Remarque

Les deux sous arbres gauches et droit sont ordonnés.

Recherche dans un arbre binaire de recherche

Fonction $\text{rechEltABR}(\text{TElement} \times \text{Arbre}) \rightarrow \text{Booléenne}$

Fonction $\text{rechEltABR}(\text{elt}, a) \rightarrow \text{booléenne}$

Raisonnement

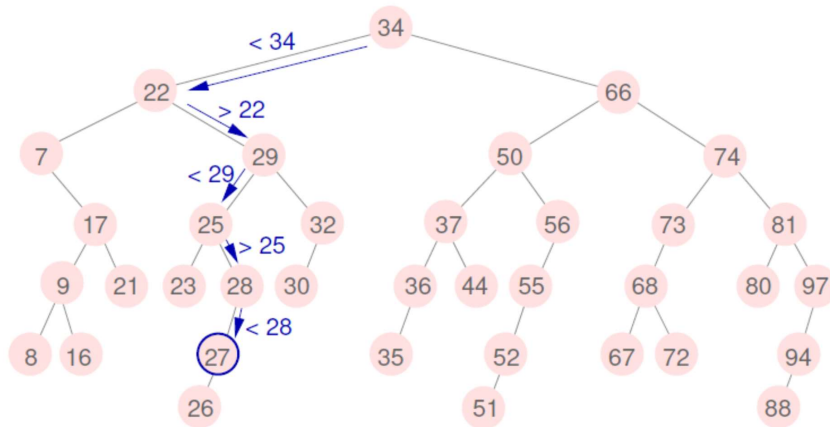
Si l'arbre est vide :

La recherche est terminée. La fonction retourne faux

Si l'arbre n'est pas vide, on distingue trois cas :

- Soit la donnée de la racine de l'arbre est égale à l'élément recherché auquel cas la recherche est terminée. La fonction retourne vraie.
- Soit la donnée de la racine est inférieure à l'élément recherché auquel cas on effectue la recherche par rapport au fils droit.
- Soit la donnée de la racine est Supérieure à l'élément recherché auquel cas on effectue la recherche par rapport au fils gauche.

Exemple



recherche de la valeur 27

Fonction rechEltABR(elt, a) → Booléenne

P.F elt : type TElement(E)

a : type arbre (E).

Début

Si estVideArbre(a) alors

Renvoyer faux

Sinon

Si donnee(a)=elt alors

Renvoyer Vrai

Sinon

Si elt < donnee(a)

Renvoyer rechEltArbreOrdonné(elt, filsGauche(a))

sinon

Renvoyer rechEltArbreOrdonné(elt, filsDroit(a))

finsi

finsi

finsi

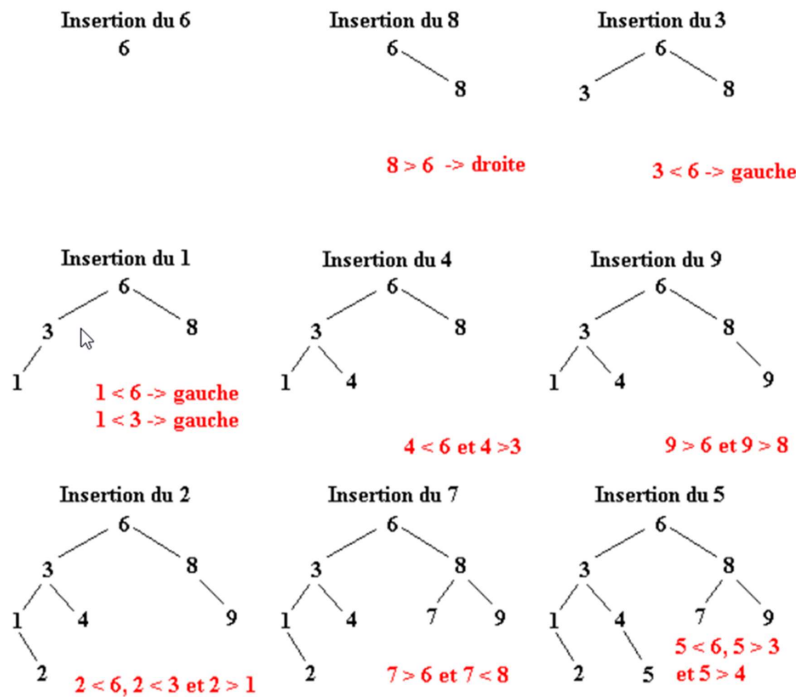
FIN

Exemple : construction d'un ABR

Construction d'un arbre binaire de recherche à partir de la liste d'entiers :

[6, 8, 3, 1, 4, 9, 2, 7, 5]

On définit la construction d'un arbre binaire de recherche par **l'insertion en feuille successive** en respectant la propriété d'ABR.



Insertion dans un arbre binaire ordonné

Problème

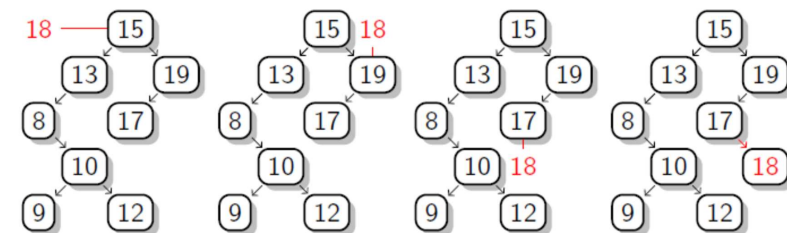
On souhaite insérer un élément donné dans un arbre ordonné tel qu'il demeure ordonné.

Fonction inserABR : TELEMENT x Arbre -> Arbre

Méthode 1 : insertion en feuille

Si l'arbre est vide, il suffit de créer une feuille contenant l'élément à insérer. Cette feuille deviendra la racine de l'arbre.

Sinon, on distingue deux cas :



- Soit la donnée de la racine est inférieure à l'élément recherché auquel cas on effectue l'insertion par rapport au fils droit.
- Soit la donnée de la racine est supérieure ou égale à l'élément recherché auquel cas on effectue l'insertion par rapport au fils gauche.

Procédure inserABR(elt , racine)

PF elt : TElement (E)

racine : Arbre (E/S)

DEBUT

Si videArbre(racine) alors

racine $\leftarrow$ creatFeuille(elt)

Sinon

Si elt > donnee(racine) alors

insérer(elt, filsDroit(racine))

Sinon // elt <= donnee(racine)

insérer(elt, filsGauche(racine))

Finsi

Finsi

Fin

ATTENTION : TRADUCTION EN C

Méthode 2 : version non récursive d'insertion en feuille

Méthode

Si l'arbre est vide, il suffit de créer une feuille contenant l'élément à insérer.

Sinon, on parcourt l'arbre, afin de rechercher un nœud qui sera le père de l'élément que nous désirons insérer.

Une fois le nœud père trouvé, il suffit de créer une feuille contenant l'élément à insérer, et l'attacher du bon côté du nœud père.

Principe

- Trouver la place de l'insertion
- Insérer l'élément :
 - Création d'une feuille et remplissage
 - Attachement (réalisation des chaînages)

// Version non récursive d'insertion d'un élément dans un arbre binaire ordonné

```
procédure inserABR(elt, a)
```

P.F elt : TElement (E)

a : Arbre (E/S)

Début

```
Si estVideArbre(a) alors
```

```
    a ← creatFeuille(elt)
```

```
Sinon
```

```
    // Déterminer l'adresse d'insertion
```

```
    adrPere ← adrInserABR(elt, a)
```

```
    // On réalise l'insertion
```

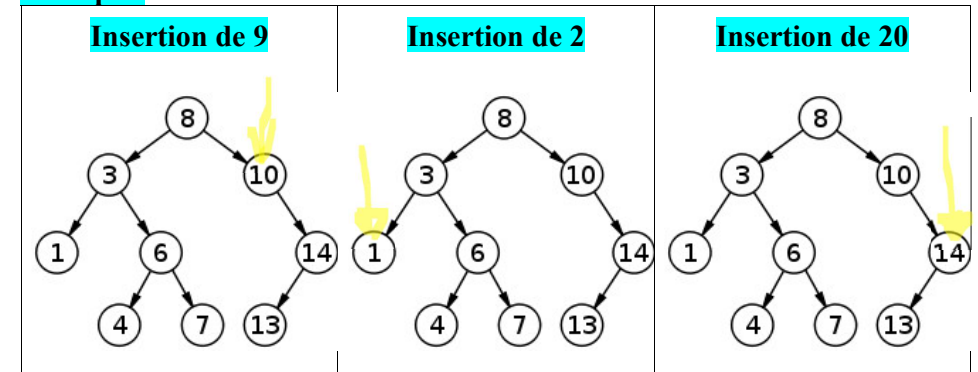
```
    insertion(elt, AdrPere)
```

```
finsi
```

Fin

Recherche de la Place d'insertion

Exemples



On recherche un nœud dans l'arbre, qui sera le père de l'élément que nous désirons insérer.

Quand nous l'aurons inséré ; ce nœud **père** est tel que :

donnee(Père) < elt et estVideArbre (FilsDroit(père))

Ou donnee(Père) >=elt et estVideArbre (FilsGauche(père))

Ce nœud père est toujours au bout de la filiation.

C'est soit une feuille soit un nœud qui n'a qu'un seul fils.

Schéma récursif de la Recherche de la Place d'insertion

On suppose que l'arbre n'est pas vide.

On distingue deux cas :

- Soit l'élément à insérer est inférieur ou égale ($\leq$) à la donnée de la racine de l'arbre $\rightarrow$ dans ce cas, l'élément fait donc partie du sous arbre gauche, on a encore deux cas :
 - Si le fils gauche est vide, on a trouvé donc le nœud qui sera le père de l'élément à insérer.
 - Si le fils gauche n'est pas vide, la place recherchée en effectuant un appel récursif sur le fils gauche.
- Soit l'élément à insérer est supérieur ($>$) à la donnée de la racine de l'arbre $\rightarrow$ dans ce cas, l'élément fait donc partie du sous arbre droit, on a encore deux cas :
 - Si le fils droit est vide, on a trouvé donc le nœud qui sera le père de l'élément à insérer.
 - Si le fils droit n'est pas vide, la place recherchée en effectuant un appel récursif sur le fils droit.

Fonction adrInserABR : Telement x Arbre $\rightarrow$ Arbre

Fonction adrInserABR (elt, a) $\rightarrow$ Arbre

P.F elt : type Telement (E). a ; type Arbre (E)

Début

Si elt $\leq$ donnee(a) alors

Si estVideArbre(filsGauche(a)) alors

Renvoyer a

Sinon

Renvoyer adrInserABR(elt, filsGauche(a))

Finsi

Sinon // elt > donnee(a)

Si estVideArbre(filsDroit(a)) alors

Renvoyer a

Sinon

Renvoyer adrInserABR(elt, filsDroit(a))

Finsi

Finsi

Fin

Algorithme réalisant l'insertion**Fonction insertion TElement x Arbre → Arbre****Procédure insert(elt, adrPere)****P.F** **elt : TElement (E)** **adrPere : Arbre(E/S)****Debut**

f ← creatFeuille(elt)

Si elt ≤ donnee(adrPere)

adrPere.filsG ← f

Sinon

adrPere.filsD ← f

Finsi

Fin**Méthode 2 : Insertion à la racine****Intérêt :**

L'adjonction à la racine peut présenter un intérêt si on désire privilégier l'accès au(x) dernier(s) élément(s).

Principe :

En fonction de l'élément (**elt**) à insérer ; cet algorithme d'insertion se décompose en deux étapes :

Première étape :

On sépare l'arbre de recherche en deux sous arbres binaires de recherches :

- Un arbre ordonné gauche qui contient les données de l'arbre qui sont inférieures ou égales (**≤**) à l'élément (**elt**)
- Un arbre ordonné droit qui contient les données de l'arbre qui sont supérieur (**>**) à l'élément (**elt**)

Puis

Deuxième étape :

Rassembler les deux sous arbres créés autour d'un nouveau (élément) racine contenant l'élément à insérer.

Découpage de l'arbre ordonné en deux arbres ordonnés gauche et droite.

Fonction séparation : TElement x Arbre $\rightarrow$ Arbre x Arbre

Données

elt : TElement // l'élément à insérer

a : arbre // ABR à découper

Traitement

Séparation de l'ABR en deux sous binaires de recherche : aG et aD :

aG contient tous les nœuds de l'arbre dont les données sont $\leq$ à elt.

aD contient tous les nœuds de l'arbre dont les données sont $>$ à elt.

On parcourt seulement les nœuds qui sont situés sur le chemin C de recherche de l'élément à insérer « elt » dans l'arbre « a » :

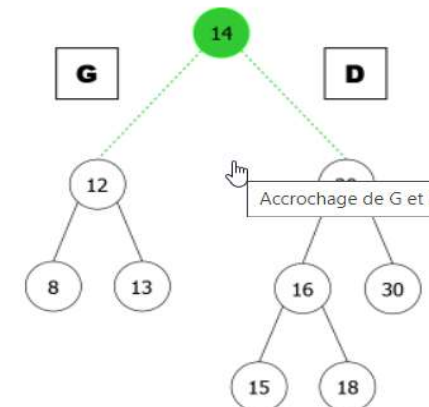
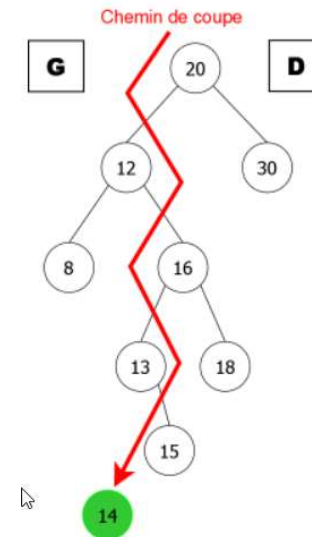
- Si un nœud de C contient un élément plus petit ou égale que l'élément à insérer, il faut alors le placer sur le fils droit de l'arbre aG.
- Si un nœud de C contient un élément plus grand que l'élément à insérer, il faut alors le placer sur le fils gauche de l'arbre aD.

Résultats

a, aG , aD : Arbre

Note : l'arbre a est modifié.

Exemple d'insertion en racine de 14



Remarques

L'insertion aux feuilles dans un ABR → entraine la modification de la hauteur de l'ABR.

L'insertion racine dans un ABR → entraine la modification de la structure de l'ABR.

procédure separation(elt, a, aG, aD)

P.F elt : TElement (E)

a : Arbre (E/S)

aG, aD : Arbre (S)

Debut

Si estVide(a) alors

aG ← initArbre()

aD ← initArbre()

Sinon

Si elt ≤ donnee(a) alors

aD ← a

Separation(elt, filsGauche(a), aG, filsGauche(aD))

Sinon

aG ← a

Separation(elt, filsDroit(a), filsDroit(aG), aD))

Fsi

Fsi

Fin

ATTENTION TRADUCTION EN C

Rassemblement des deux sous ABR-Gauche et ABR-Droit autour de l'élément « elt »

Données

elt : TElement // l'élément à insérer

a : Arbre

aG : Arbre

AD : Arbre

Traitement

$A \leftarrow \text{elt} + \text{aG} + \text{aD}$

Résultats

a : arbre

procédure inserRacineEltABR(elt, a)

P.F elt : TElement (E)

a : Arbre (E/S)

Debut

//Création feuille contenant l'élément à insérer

$f \leftarrow \text{creatFeuille}(\text{elt})$

// On effectue la séparation

séparation(elt, a, filsGauche(f), filsDroit(f))

//Résultat

$a \leftarrow f$

Fin

ATTENTION TRADUCTION EN C

Application : tri d'un tableau

Principe

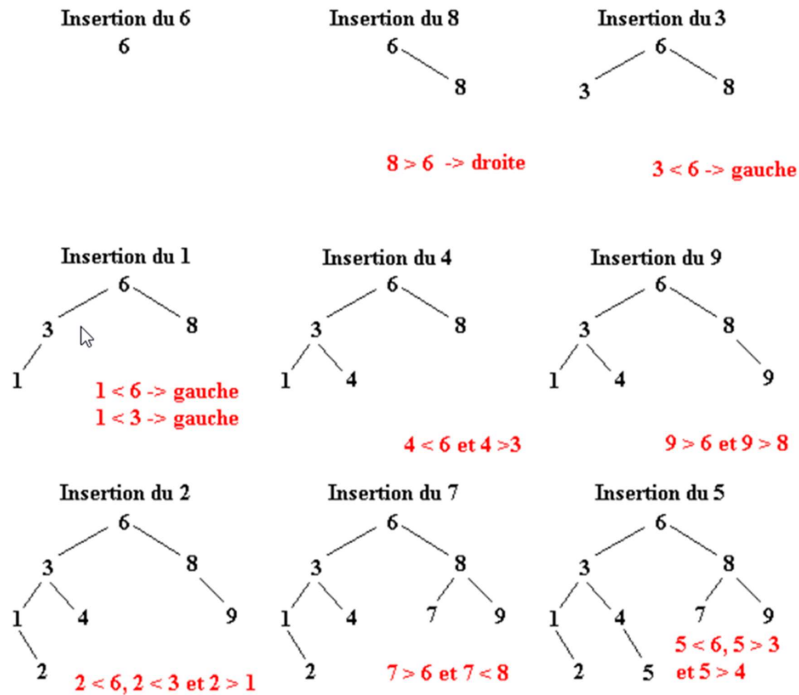
Pour trier un tableau, on procède comme suit :

- Création d'un arbre binaire ordonné contenant toutes les valeurs du tableau.
- Effectuer un parcours infixe de l'arbre en rangeant les éléments de l'arbre dans le tableau afin d'obtenir un tableau trié.
- Libération de la mémoire occupée par l'arbre binaire ordonné

Exemple : tri du tab=[6, 8, 3, 1, 4, 9, 2, 7, 5]

1) Construction d'un arbre binaire de recherche à partir de la liste d'entiers :

On définit la construction d'un arbre binaire de recherche par l'insertion successive en respectant la propriété d'ABR.



2) Effectuer un parcours infixe de l'arbre en rangeant les éléments de l'arbre dans le tableau afin d'obtenir un tableau trié.

Tab=[1, 2, 3, 4, 5, 6, 7, 9]

Algorithme de tri

Fonction triParABR : Entier x Tableau $\rightarrow$ Tableau

procédure triParABR(n, tab)

PF n : Entier (E)

tab ; Tableau (E/S)

Début

// Création de l'ABR

racine $\leftarrow$ creatFeuille(tab[1])

Pour i variant de 2 à n faire

 inserABR(tab[i], racine)

Fpour

// Parcours Infixe de l'ABR

n $\leftarrow$ 1

parcoursInfixeABRTab(n, racine, tab)

// Libération de la mémoire occupée par l'arbre binaire ordonné

suppressionArbre(racine)

Fin

Rappel Parcours Infixe

Ce parcours consiste à effectuer dans l'ordre :

- d) Parcours du fils gauche
- e) Traitement de la racine
- f) Parcours du fils droit

Procédure `parcoursInfixeABRTab(n, racine, tab)`

PF `n` : Entier (E/S)

`racine` : Arbre (E)

`tab` : Tableau (E/S)

Début

Si `non estVideArbre(racine)` alors

`parcoursInfixeABRTab(n, filsGauche(racine), tab)`

// Traitement de la racine

`tab[n] ← donnee(racine)`

`n ← n+1`

`parcoursInfixeABRTab(n, filsDroit(racine), tab)`

FinSi

Fin

Complexité

Pour parcourir l'arbre binaire de recherche, la complexité est proportionnelle à la taille de l'arbre, c'est à dire le nombre d'éléments que l'arbre contient. Soit une complexité en $O(n)$ où n est le nombre d'éléments à classer.

Pour construire l'arbre binaire de recherche, il faut insérer les n éléments un à un dans un arbre initialement vide.

Pire des cas

Dans le pire des cas, l'arbre va se construire sous forme d'une liste (arbre filiforme), c'est le cas, par exemple, lorsque l'ensemble des valeurs est initialement classé à l'envers.

Dans ce cas, pour classer le $p^{\text{ième}}$ élément, il faut effectuer $(p-1)$ comparaisons.

$$\sum_{p=2}^n (p-1) = \frac{1}{2}n^2 - \frac{1}{2}n$$

Soit une complexité $T(n)$ donnée par la formule suivante :

Ce qui correspond à une complexité dans le pire des cas en $O(n^2)$.

Meilleur des cas

Dans le meilleur des cas, la complexité de ce tri est heureusement bien meilleure. Par contre, elle est un peu plus compliquée à calculer.

Le meilleur des cas correspond au cas où l'arbre est complet une fois construit. Cela correspond à un arbre de hauteur h et dont toutes les branches de profondeur $(h-1)$ possèdent un arbre droit et un arbre gauche. Tous les niveaux de l'arbres sont donc bien remplis. Dans ce cas l'arbre est capable de stocker le nombre d'éléments suivant :

Profondeur Nombres d'éléments stockés

0	1
1	1+2 = 3
2	1+2+4 = 7
j	1+2+4+2 ³ +...+2 ^j

Quand le j^{ème} niveau est rempli, il y a donc le nombre d'éléments suivant dans

$$\sum_{i=0}^j 2^i = 2^{(j+1)} - 1$$

l'arbre : $i=0$

Dans le meilleur des cas, pour insérer le p^{ème} élément, il faut effectuer le nombre de comparaisons correspondant au nombre de niveaux remplis.

Si le p^{ème} élément est inséré au niveau j, on a $p=2^{j+1}$, d'où $p-1=2^{j+1}-1$, soit $j+1=\lg(p-1)$ avec $\lg$ le logarithme népérien en base 2.

Soit, pour finir, $j=\lg(p-1)-1$.

Ce résultat correspond au nombre de comparaisons pour insérer le p^{ème} élément.

Pour insérer les n éléments de l'ensemble à classer dans l'arbre, et donc pour construire ce même arbre, il faut effectuer le nombre de comparaisons T(n)

$$T(n) = \sum_{p=1}^n \left(\frac{\ln(p+1)}{\ln(2)} - 1 \right)$$

suivant :

$$T(n) = \left(\sum_{p=1}^n \frac{\ln(p+1)}{\ln(2)} \right) - n$$

Ce qui se simplifie ainsi :

$$T(n) = \frac{\ln \left(\prod_{p=1}^n (p+1) \right)}{\ln(2)} - n$$

Or, $\ln(a)+\ln(b)=\ln(a.b)$, d'où le résultat suivant :

Ce qui s'écrit encore plus simplement : $T(n) = \frac{\ln((n+1)!)}{\ln(2)} - n$

Il s'agit du calcul exact de la complexité pour construire un arbre binaire de recherche complet à n éléments.

Cependant, ce calcul est assez difficile à exploiter : est-ce que ce tri est plus avantageux que les tris vu précédemment ? Nous allons montrer ici qu'il s'agit d'un tri en $O(n.\ln(n))$.

Pour cela, il faut montrer l'égalité suivante :

$$\lim_{n \rightarrow \infty} \frac{T(n)}{n \ln(n)} = \text{constante}$$

$$f(n) = \frac{\frac{\ln((n+1)!)}{\ln(2)} - n}{n \ln(n)}$$

On pose :

$$\text{Ce qui se simplifie ainsi : } f(n) = \frac{\ln((n+1)!)}{\ln(2) n \ln(n)} - \frac{1}{\ln(n)}$$

$$\text{Or, on a : } \lim_{n \rightarrow \infty} \frac{1}{\ln(n)} = 0$$

$$\text{donc, on a : } \lim_{n \rightarrow \infty} f(n) = \lim_{n \rightarrow \infty} \frac{\ln((n+1)!)}{\ln(2) n \ln(n)}$$

Pour calculer cette limite, il faut en chercher un équivalent à l'infini. Or, d'après

la formule de Stirling, on a n! qui est équivalent à l'infini à : $\left(\frac{n}{e}\right)^n \sqrt{2\pi n}$

$$\frac{\ln\left(\left(\frac{n}{e}\right)^n \sqrt{2} \sqrt{n \pi}\right)}{\ln(2) n \ln(n)}$$

Soit $f(n)$ qui est équivalent à l'infini à :

$$\frac{n \ln(n) - n + \frac{1}{2} \ln(2 \pi) + \frac{1}{2} \ln(n)}{\ln(2) n \ln(n)}$$

Expression qui se simplifie ainsi :

$$\text{Soit encore : } \frac{1}{\ln(2)} - \frac{1}{\ln(n)} + \frac{1}{2} \frac{\ln(2 \pi)}{\ln(2) n \ln(n)} + \frac{1}{2} \frac{1}{n}$$

Or, les trois derniers termes de cette expression tendent de manière évidente vers 0.

$$\lim_{n \rightarrow \infty} f(n) = \frac{1}{\ln(2)}$$

D'où le résultat :

$$\lim_{n \rightarrow \infty} \frac{T(n)}{n \ln(n)} = \text{constante}$$

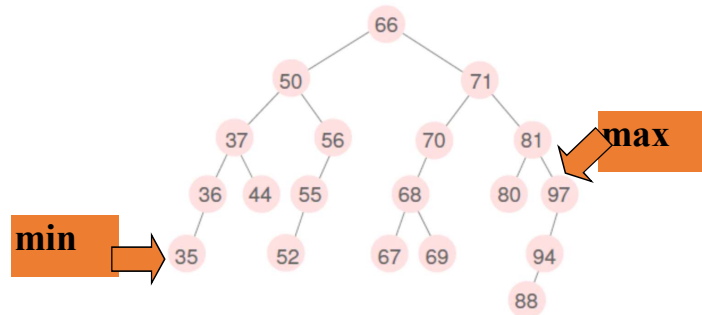
CQFD : C'est ce qu'il fallait démontrer :

D'où $T(n) = O(n \ln(n))$, et la complexité de la construction d'un arbre binaire de recherche complet est en $O(n \ln(n))$.

Conclusion : Le tri par Arbre Binaire de Recherche a les mêmes caractéristiques que le tri rapide.

Le seul inconvénient de ce tri est qu'il nécessite la construction d'un arbre binaire de recherche, qui requiert un espace mémoire suffisamment important. Le tri rapide, lui, s'effectue en utilisant l'espace mémoire utilisé par le tableau à trier, il ne nécessite donc pas d'espace mémoire supplémentaire pour construire une nouvelle structure assez compliquée.

Le tri fusion, quant à lui, utilise également un espace mémoire important, mais sa complexité en $O(n \ln(n))$ est garantie, quel que soit l'ensemble à trier.

Algorithmes de Recherche du plus petit**(resp. Plus Grand) élément dans un ABR****Principe**

Le **plus petit élément** dans un ABR est l'élément le plus à gauche. C'est-à-dire le nœud le plus à gauche qui ne possède pas de fils gauche

Le **plus grand élément** dans un ABR est l'élément le plus à droite. C'est-à-dire le nœud le plus à droite qui ne possède pas de fils droit.

Précondition : L'arbre est non vide**Rechercher l'élément le plus à gauche****Principe :**

Le **plus petit élément** dans un ABR est l'élément le plus à gauche. C'est-à-dire le nœud le plus à gauche qui ne possède pas de fils gauche

Version récursive**Fonction minABR : Arbre → TElement**

Fonction minABR(racine) --> TElement

P.F racine : Arbre (E)

Début

fg <-- filsGauche(racine)

si estVide(fg) alors

renvoyer donnee(racine)

sinon

renvoyer minABR(fg)

fsi

Fin

Version itérative

Fonction minABR : Arbre → TElement

Fonction minABR(racine) --> TElement

P.F racine : Arbre (E)

Début

fg <-- filsGauche(racine)

tantque non estVide(fg) faire

 racine ← fg

 fg <-- filsGauche(racine)

finTq

renvoyer donnee(racine)

FIN

Recherche l'adresse du plus petit (plus grand) élément dans un ABR

Précondition : L'arbre est non vide

Principe :

Le **plus petit élément** dans un ABR est l'élément le plus à gauche. C'est-à-dire le nœud le plus à gauche qui ne possède pas de fils gauche

Version récursive

Fonction minABR : Arbre → Arbre

Fonction minABR(racine) --> Arbre

Pré- condition : L'arbre est non vide

P.F racine : Arbre (E)

Début

fg <-- filsGauche(racine)

si estVide(fg) alors

 renvoyer racine

sinon

 renvoyer minABR(fg)

fsi

Fin

Version itérative

Fonction minABR : Arbre → Arbre

Fonction minABR(racine) --> TElement

P.F racine : Arbre (E)

Début

```
fg <-- filsGauche(racine)
```

```
tantque non estVide(fg) faire
```

```
    racine ← fg
```

```
    fg <-- filsGauche(racine)
```

```
finTq
```

```
renvoyer racine
```

FIN

Algorithmes de suppressions

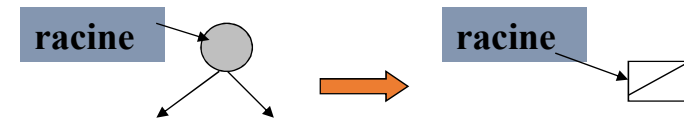
Suppression du nœud racine

Hypothèse : non estVide(a)

Principe

Suppression de la racine on distingue les cas suivants :

a) si l'arbre est réduit à une feuille, on supprime la feuille.



b) Sinon, on remplace la racine par son fils droit s'il n'est pas vide et on rattache le fils gauche de la racine au plus petit élément de son fils droit.



Par contre on remplace la racine par son fils gauche si le fils droit de l'arbre est vide.



Remarque : Souvent la forme de l'ABR est complètement modifiée

Exemples

Version1

Hypothèse : non estVide(a)

Fonction SuppRacineABR : Arbre → Arbre

Procédure SuppRacineABR(racine)

PF racine : Arbre (E/S)

Début

//Préservation du nœud à supprimer

rr ← racine

// Modification des chaînages

si estFeuille(racine) alors

racine ← initA()

else

fg <-- filsGauche(racine)

fd <-- filsDroit(racine)

Si non estVideArbre(fd) alors

//Recherche du Plus Petit Nœud de FD

ppndfd ← fd

Tantque non estVide(filsGauche(ppndfd))

faire

ppndfg ← filsGauche(ppndfd)

Fintq

ppndfd.filsg ← fg

racine ← fd

Sinon

racine ← fg

Finsi

Finsi

//libération Mémoire

LibMémoire(rr)

Fin

Version2 suppression racine**Procédure SuppRacineABR(racine)**

PF racine : Arbre (E/S)

Début

```
// Préservation du nœud racine
rr ← racine
```

```
// Modification chaînage
```

```
si estFeuille(racine) alors
    racine ← initArbre()
sinon
```

```
    fg <-- filsGauche(racine)
    fd <-- filsDroit(racine)
    Si non estVideArbre(fd) alors
        //Recherche du Plus Petit Nœud de FD
        ppndfd ← minABR(fd)
        ppndfd.filsg ← fg
        Racine ← fd
    Sinon
        racine ← fg
    Finsi
Finsi
```

```
// Libération Mémoire
```

```
LibMémoire(rr)
```

Fin**Suppression d'un élément dans un arbre binaire ordonné****Fonction suppEltABR : TElement x Arbre → Arbre****Précondition : non estVide(a)****Problème**

On se propose de supprimer un élément dans un arbre binaire ordonné de telle sorte que l'arbre reste ordonné.

Méthode 1 : Même principe que la recherche dichotomique

Si l'arbre est vide → c'est que l'élément à supprimer n'appartient pas à l'ABR

Arrêt de l'algorithme

Si l'arbre est non vide : on distingue les cas suivants :

- Cas où donnée (a) = elt → on applique suppression racine
- Cas où elt < donnée(a) → on effectue la suppression par rapport au fils gauche de l'ABR.
- Cas où elt > donnée(a) → on effectue la suppression par rapport au fils droit de l'ABR.

Fonction `suppEltABR` : `TElement x Arbre → Arbre`

Procédure `suppEltABR(elt, racine)`

P.F

elt : TElement (E)

racine : Arbre (E/S)

Début

Si non `estVideArbre(racine)` alors

 Si `donnee(racine)=elt` alors

`SuppRacineABR(racine)`

 Sinon

 Si `elt < donnee(racine)` alors

`suppEltABR(elt, filsGauche(racine))`

 Sinon

`suppEltABR(elt, filsDroit(racine))`

 Finsi

 Finsi

Finsi

Fin **Attention traduction en C**

Méthode 2 : suppression d'un élément donné dans ABR**donné****Principe**

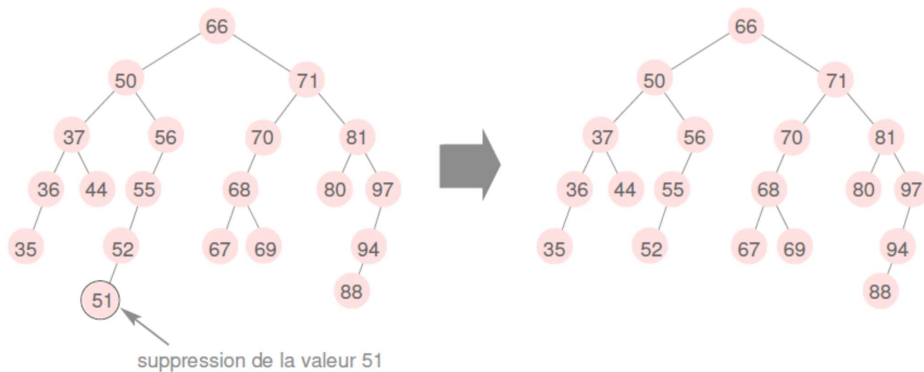
L'opération de suppression se décompose en deux étapes :

- Une étape de recherche.
- Une autre étape de suppression qui dépend de l'élément à supprimer « elt » dans l'ABR.

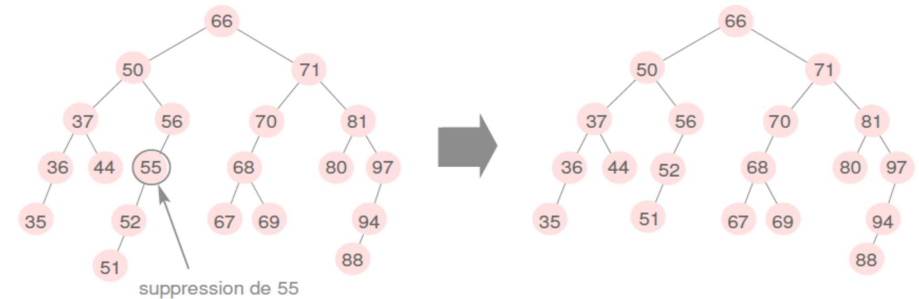
Remarque : la forme de l'ABR est légèrement modifiée

Pour cette suppression, plusieurs cas sont possibles :

- Si le nœud contenant l'élément à supprimer est une feuille, alors on supprime la feuille directement



- Si le nœud contenant l'élément à supprimer possède un seul fils, alors on remplacera le nœud par son fils, et la propriété de l'ABR ne change pas



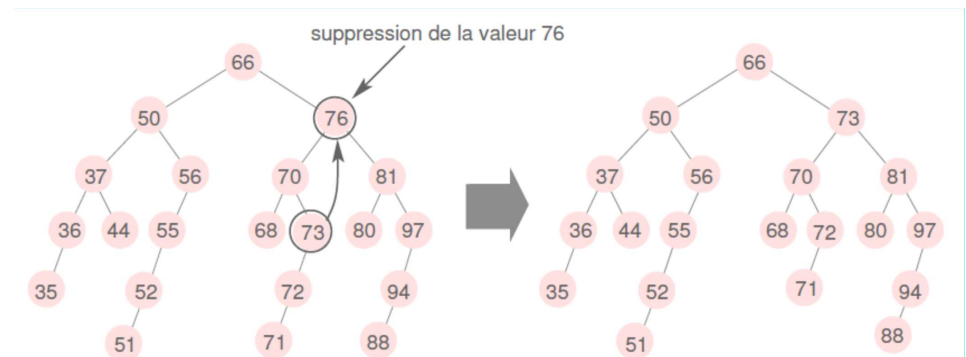
- Si le nœud contenant l'élément à supprimer possède deux fils, plusieurs solutions sont possibles :
 - On remplacera l'élément à supprimer par son **prédécesseur** ; c'est à dire le plus grand élément qui se trouve dans le sous arbre gauche du nœud contenant l'élément à supprimer.

Donnee(nœud à supprimer) $\leftarrow$ supMax(filsGauche(nœud))

Où

- On remplacera l'élément à supprimer par son **successeur** ; c'est à dire le plus petit élément qui se trouve dans le sous arbre droit du nœud contenant l'élément à supprimer.

Donnee(nœud à supprimer) $\leftarrow$ supMin(filsDroit(nœud))



Précondition : non estVideArbre(a)**Changement de la suppression racine****Procédure suppRacine_V2(racine)**

P.F

racine : Arbre (E/S)

Début

fg ← filsGauche(racine)

fd ← filsDroit(racine)

Si estVide(fg) alors

rr ← racine

racine ← fd

libMemoire(rr)

sinon

si estVide(fd) alors

rr ← racine

racine ← fg

libMemoire(rr)

sinon // deux fils

eMax ← suppMax(filsGauche(racine))

racine.donne ← eMax

fsi

fsi

FIN

Suppression de l'élément max dans un arbre binaire de recherche**Précondition : non estVide(racine)****Fonction suppMax : Arbre → TElement x Arbre**

Cet algorithme supprime le nœud contenant l'élément maximum dans un ABR (racine) donné.

Rappel : l'élément max dans un ABR est l'élément le plus à droite.

Fonction suppMax(racine) → TElement

P.F racine : Arbre (E/S)

Debut

fd ← filsDroit(racine)

Si estVide(fd) alors

eMax ← donnee(racine)

//Préserver le nœud à supprimer

rr ← racine

racine ← filsGauche(racine)

libMemoire(rr)

renvoyer eMax

sinon

renvoyer suppMax(fd)

fsi

FIN

Attention traduction en C

Remarque : la forme de l'ABR est légèrement modifiée

Fonction `suppEltABR` : `TElement x Arbre → Arbre`

Procédure `suppEltABR_V2(elt, racine)`

P.F

elt : `TElement (E)`

racine : `Arbre (E/S)`

Début

Si non `estVideArbre(racine)` alors

Si `donnee(racine)=elt` alors

//changement proc `suppRacine`

`suppRacine_V2(racine)`

Sinon

Si `elt < donnee(racine)` alors

`suppEltABR_V2(elt, filsGauche(racine))`

Sinon

`suppEltABR_V2(elt, filsDroit(racine))`

Finsi

Finsi

Finsi

Fin **Attention traduction en C**

Exercice

Vérifier si un arbre binaire donné est un arbre binaire de recherche

Rappel : Propriété d'un ABR

Un ABR est un arbre binaire $a=(racine, aG, aD)$ tel que :

aG et aD sont des ABRs

et

si aG et aD sont non vide alors `donnee(aG) <= donnee(racine) < donnee(aD)`

si aG est Vide et aD non vide alors `donne(racine) < donnee(aD)`

si aD est Vide et aG non vide alors `donnee(aG) <= donne(racine)`

Intérêts des ABR

Les Arbres Binaires de Recherche sont une structure de données qui permet de représenter correctement un ensemble ordonné de données.

Les alternatives (tableaux ou listes) ont de bien moins bons comportements dans les opérations usuelles : ajout / suppression / recherche.

Tous les algorithmes des ABRs, le temps de calcul est proportionnel à la hauteur de l'arbre (et non au nombre de données comme pour les autres alternatives)

Toutefois, dans le pire des cas, la hauteur de l'arbre est le nombre de données dans l'arbre.

En moyenne, les ABRs sont quand même souvent intéressants.

Pour éviter les mauvais cas : **équilibrage des arbres**.